

P4.py create model

```
# import lib
import pandas as pd
from nltk import word_tokenize
from string import punctuation
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report
from pickle import dump

# load data
data = pd.read_csv("fakenews.csv")
print(data)

# check and remove null data
print(data.isnull().sum())

# data cleaning
sw = stopwords.words("english")
ps = PorterStemmer()
def clean_text(txt):
    txt = txt.lower()
    txt = word_tokenize(txt)
    txt = [t for t in txt if t not in sw]
    txt = [t for t in txt if t not in punctuation]
    txt = [ps.stem(t) for t in txt]
    txt = " ".join(txt)
    return txt

data["clean_text"] = data["text"].apply(clean_text)
print(data)

# features and target
tf = TfidfVectorizer()
vector = tf.fit_transform(data["clean_text"])
features = pd.DataFrame(vector.toarray(), columns=tf.get_feature_names_out())
print(features)
target = data["label"]
print(target)

# train and test
x_train, x_test, y_train, y_test = train_test_split(features.values, target)

# model
model = MultinomialNB()
model.fit(x_train, y_train)
```



```
# classification_report
y_pred = model.predict(x_test)
cr = classification_report(y_test, y_pred)
print(cr)
```

```
# save the model and tf
with open("news_model.pkl", "wb") as f:
    dump(model, f)
    print("model saved")
with open("news_tf.pkl", "wb") as f:
    dump(tf, f)
    print("tf saved ")
```

NLP with Kamal Sir - 8369084928

P5.py **use the model**

```
# import lib
from pickle import load
from nltk import word_tokenize
from string import punctuation
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# data cleaning
sw = stopwords.words("english")
ps = PorterStemmer()
def clean_text(txt):
    txt = txt.lower()
    txt = word_tokenize(txt)
    txt = [t for t in txt if t not in sw]
    txt = [t for t in txt if t not in punctuation]
    txt = [ps.stem(t) for t in txt]
    txt = " ".join(txt)
    return txt

# get back mode and tf

# prediction
```